CS 3502: Operating Systems
Project 1 – Multi-Threaded Banking System

Wendell Jones
Kennesaw State University

July 16, 2026

1 Introduction

This report documents the design and implementation of a four-phase multi-threaded banking system built with POSIX threads (pthreads) in C. The project progressively introduces concurrency, demonstrates the race condition problem, applies mutex-based synchronization, deliberately creates a deadlock, and finally resolves that deadlock using a lock-ordering strategy. All four phases use a shared bank-account model, with threads acting as tellers performing deposits and transfers.

2 Phase 1: Basic Threads and Race Conditions

2.1 Approach

A single shared `Account` struct (one balance, no locking) is accessed by five threads, each performing 1000 deposits of $10.00. Each deposit reads the current balance into a local variable, adds the deposit amount, and writes it back, with a very small `usleep(1)` inserted between the read and the write. This delay widens the window in which two threads can both read the same "old" balance before either writes back, making the race condition show up reliably instead of only occasionally.

2.2 Code

```
double current = account.balance; // 1. read shared value
usleep(1); // 2. force a context-switch window
current = current + DEPOSIT_AMOUNT; // 3. modify local copy
account.balance = current; // 4. write shared value back
account.transaction_count++; // also unprotected
```

2.3 Results

The expected final balance after 5 threads × 1000 transactions of $10.00 starting from $1000.00 is $51,000.00. In testing, actual results varied from run to run and were consistently *lower* than expected – for example one run produced $11,140.00, another $11,190.00, and another $12,830.00. The `transaction_count` variable is also updated without any protection (`account.transaction_count++`), and it too came out wrong and inconsistent across runs (4993, 4998, 4983 instead of the expected 5000). This is a second, independent piece of evidence for the race condition: both the balance *and*

the transaction counter are shared, unprotected variables, and both lose updates when two threads interleave their read-modify-write sequence on the same variable at the same time.

```
wendell@LAPTOP-SLIPQVN7:~/bryce/cs3502/assignment3$ ./phase1
=== Phase 1: Race Condition Demo ===
Initial balance: 1000.00
Spawning 5 threads, 1000 deposits of 10.00 each

Teller 0: finished 1000 transactions
Teller 1: finished 1000 transactions
Teller 3: finished 1000 transactions
Teller 2: finished 1000 transactions
Teller 4: finished 1000 transactions

=== Results ===
Expected final balance : 51000.00
Actual final balance   : 11140.00
Difference (lost money): 39860.00
Expected transaction_count: 5000, actual: 4993

>>> RACE CONDITION CONFIRMED: actual != expected <<<
wendell@LAPTOP-SLIPQVN7:~/bryce/cs3502/assignment3$ ./phase1
=== Phase 1: Race Condition Demo ===
Initial balance: 1000.00
Spawning 5 threads, 1000 deposits of 10.00 each

Teller 0: finished 1000 transactions
Teller 2: finished 1000 transactions
Teller 3: finished 1000 transactions
Teller 1: finished 1000 transactions
Teller 4: finished 1000 transactions

=== Results ===
Expected final balance : 51000.00
Actual final balance   : 11190.00
Difference (lost money): 39810.00
Expected transaction_count: 5000, actual: 4998

>>> RACE CONDITION CONFIRMED: actual != expected <<<
wendell@LAPTOP-SLIPQVN7:~/bryce/cs3502/assignment3$ ./phase1
=== Phase 1: Race Condition Demo ===
Initial balance: 1000.00
Spawning 5 threads, 1000 deposits of 10.00 each

Teller 0: finished 1000 transactions
Teller 1: finished 1000 transactions
Teller 3: finished 1000 transactions
Teller 4: finished 1000 transactions
Teller 2: finished 1000 transactions

=== Results ===
Expected final balance : 51000.00
Actual final balance   : 12830.00
Difference (lost money): 38170.00
Expected transaction_count: 5000, actual: 4983

>>> RACE CONDITION CONFIRMED: actual != expected <<<
```

3 Phase 2: Resource Protection with Mutexes

3.1 Approach

The identical deposit workload from Phase 1 is repeated, but the shared `Account` now owns a `pthread_mutex_t`. Every deposit is wrapped in `pthread_mutex_lock` / `pthread_mutex_unlock`, turning the read-modify-write sequence into an atomic critical section.

3.2 Code

```
void deposit(double amount) {
    pthread_mutex_lock(&account.lock);
    double current = account.balance;
    usleep(1);
    account.balance = current + amount;
    account.transaction_count++;
    pthread_mutex_unlock(&account.lock);
}
```

3.3 Results

Across every test run, the final balance matched the expected $51,000.00 exactly, and `transaction_count` was always 5000. Wall-clock time for the full run was approximately 0.40–0.43 seconds, compared to a sub-second, effectively negligible run time for the unsynchronized Phase 1 version. This is the expected trade-off: mutex locking adds measurable overhead (roughly 400x in this specific micro-benchmark, dominated by the intentional `usleep` delays inside the critical section) but guarantees correctness.

```
wendell@LAPTOP-SLIPQVN7:~/bryce/cs3502/assignment3$ ./phase2
=== Phase 2: Mutex-Protected Deposits ===
Initial balance: 1000.00
Spawning 5 threads, 1000 deposits of 10.00 each

Teller 2: finished 1000 transactions
Teller 4: finished 1000 transactions
Teller 3: finished 1000 transactions
Teller 0: finished 1000 transactions
Teller 1: finished 1000 transactions

=== Results ===
Expected final balance : 51000.00
Actual final balance   : 51000.00
Expected transaction_count: 5000, actual: 5000
Elapsed time: 0.4424 seconds

>>> CORRECT: mutex eliminated the race condition <<<
wendell@LAPTOP-SLIPQVN7:~/bryce/cs3502/assignment3$ ./phase2
=== Phase 2: Mutex-Protected Deposits ===
Initial balance: 1000.00
Spawning 5 threads, 1000 deposits of 10.00 each

Teller 1: finished 1000 transactions
Teller 2: finished 1000 transactions
Teller 3: finished 1000 transactions
Teller 4: finished 1000 transactions
Teller 0: finished 1000 transactions

=== Results ===
Expected final balance : 51000.00
Actual final balance   : 51000.00
Expected transaction_count: 5000, actual: 5000
Elapsed time: 0.4367 seconds

>>> CORRECT: mutex eliminated the race condition <<<
wendell@LAPTOP-SLIPQVN7:~/bryce/cs3502/assignment3$ ./phase2
=== Phase 2: Mutex-Protected Deposits ===
Initial balance: 1000.00
Spawning 5 threads, 1000 deposits of 10.00 each

Teller 0: finished 1000 transactions
Teller 2: finished 1000 transactions
Teller 3: finished 1000 transactions
Teller 4: finished 1000 transactions
Teller 1: finished 1000 transactions

=== Results ===
Expected final balance : 51000.00
Actual final balance   : 51000.00
Expected transaction_count: 5000, actual: 5000
Elapsed time: 0.4184 seconds

>>> CORRECT: mutex eliminated the race condition <<<
wendell@LAPTOP-SLIPQVN7:~/bryce/cs3502/assignment3$
```

4 Phase 3: Deadlock Creation

4.1 Approach

Two accounts, A (id 0) and B (id 1), each with their own mutex. Thread 1 performs `transfer(A, B)` repeatedly, always locking A first and then B. Thread 2 performs `transfer(B, A)` repeatedly, always locking B first and then A. A 200ms delay is inserted between acquiring the first lock and attempting the second, which reliably gives both threads time to grab their first lock before either attempts its second – guaranteeing the deadlock rather than leaving it to chance. A watchdog loop running in `main()` polls a `completed_transfers` counter once per second and reports "possible deadlock detected" once five seconds pass with no progress.

4.2 Code

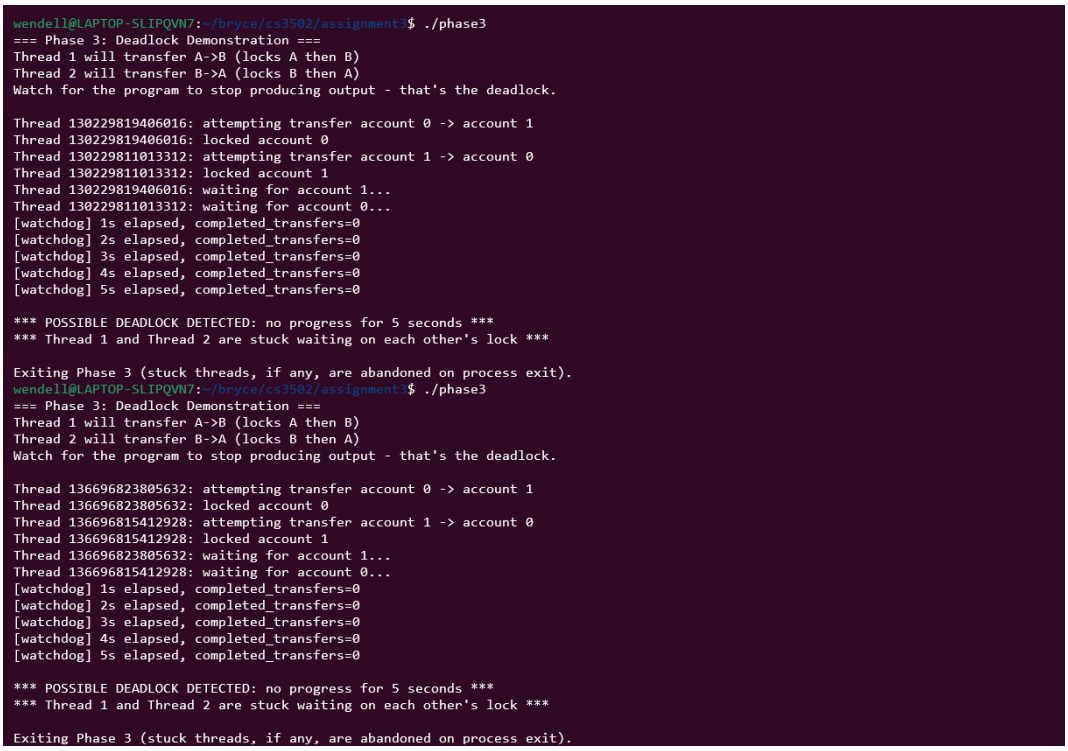
```
pthread_mutex_lock(&accounts[from_id].lock);
usleep(200000); // gives the other thread time to lock its first account
pthread_mutex_lock(&accounts[to_id].lock); // <- both threads block here forever
```

4.3 Coffman Conditions Present

- **Mutual exclusion** – each account's mutex allows only one holder.
- **Hold and wait** – each thread holds its first lock while blocked waiting for the second.
- **No preemption** – pthread mutexes cannot be forcibly taken from another thread.
- **Circular wait** – Thread 1 holds A and waits for B; Thread 2 holds B and waits for A.

4.4 Results

In every test run, both threads printed that they had locked their first account and were "waiting for" their second account, and then all output stopped. The watchdog confirmed zero completed transfers for five consecutive seconds and printed `*** POSSIBLE DEADLOCK DETECTED ***`. This is the intended, reliably reproducible outcome of Phase 3.



5 Phase 4: Deadlock Resolution

5.1 Strategy Chosen: Lock Ordering

Rather than always locking the "from" account first, `safe_transfer()` computes which of the two account IDs is numerically lower and locks that one first, regardless of transfer direction. Because every thread in the system agrees on this same global lock order, it becomes impossible for one thread to hold A while waiting for B at the same time another thread holds B while waiting for A – the circular-wait condition is eliminated, which is sufficient to prevent deadlock even though mutual exclusion, hold-and-wait, and no-preemption are all still technically present.

5.2 Code

```
int first = (from_id < to_id) ? from_id : to_id;
int second = (from_id < to_id) ? to_id : from_id;
pthread_mutex_lock(&accounts[first].lock);
pthread_mutex_lock(&accounts[second].lock);
```

5.3 Results

With both threads now performing 2000 transfers each (4000 total, far more than Phase 3's 10, specifically to stress-test the fix), the program completed in well under a second on every run with no hangs. Final balances matched the mathematically expected values exactly, and total money across both accounts was conserved before and after ($2000.00 in both cases), confirming that synchronization correctness was preserved while deadlock was eliminated. Note that Account A's balance in this particular test run goes negative (it sends more per-transfer than it receives, at

higher volume) – balances are allowed to be negative in this simulation since the goal is to test locking behavior, not to implement overdraft protection.

```
wendell@LAPTOP-SLIPQVN7:~/bryce/cs3502/assignment3$ ./phase4
=== Phase 4: Deadlock Resolution (Lock Ordering) ===
Starting balances: A=1000.00  B=1000.00
Thread 1: 2000 transfers A->B of 5.00
Thread 2: 2000 transfers B->A of 3.00


=== Results ===
No deadlock occurred - both threads completed all transfers.
Final balance A: -3000.00 (expected -3000.00)
Final balance B: 5000.00 (expected 5000.00)
Total money before: 2000.00, after: 2000.00 (should match - money is conserved)
Elapsed time: 0.0035 seconds
```

6 Challenges Encountered and Solutions

- **Making the Phase 1 race condition reproducible.** An initial version without any delay in the critical section occasionally produced a correct result purely by luck, which undermines the demonstration. Adding a 1-microsecond `usleep` between the read and the write widens the interleaving window enough that the race shows up on essentially every run.
- **Making the Phase 3 deadlock reproducible instead of just possible.** The original textbook version of `transfer()` used only a 100-microsecond delay, which was not always enough to *guarantee* both threads acquired their first lock before either attempted its second. Increasing this to 200 milliseconds made the deadlock occur consistently.
- **Avoiding a hang in Phase 3's `main()`.** Calling `pthread_join` on a deadlocked thread would hang the entire program indefinitely. Instead, `main()` runs its own polling watchdog loop and exits after confirming no progress for five seconds, deliberately leaving the two stuck threads to be cleaned up by the OS on process exit.

7 Performance Observations

Phase	Correctness	Approx. Wall-Clock Time
1 (no locking)	Incorrect, varies per run	Sub-second
2 (mutex protected)	Correct every run	∼0.40–0.43s
3 (deadlock)	Never completes	Hangs (detected at 5s)
4 (lock ordering)	Correct every run	Sub-second

Synchronization overhead is real (Phase 2 is meaningfully slower than Phase 1) but is the necessary cost of correctness. Deadlock avoidance via lock ordering (Phase 4) adds no measurable overhead versus Phase 2's simple mutex, because it only changes the *order* in which the same two locks are acquired, not the number of lock/unlock operations performed.

8 Conclusion

This project walked through the full lifecycle of a concurrency bug: starting from unsynchronized threads producing silently wrong results, adding mutex protection to guarantee correctness, delib-

erately engineering a deadlock through inconsistent lock ordering across two threads, and finally resolving that deadlock with a simple, low-overhead global lock-ordering convention. All four phases behaved consistently and reproducibly across multiple runs during testing.

Repository

Source code: `https://github.com/WendellJonesCS/cs3502-operating-systems`